

Laboratory 0x03

Expected delivery of lab_03.zip must include:

- program_1.s, program_1_a.s, and program_1_b.s
- This file, filled with information and possibly compiled in a PDF format.

Delivery date:

October 30th 2025

This lab will explore some of the concepts seen during the lessons, such as hazards, rescheduling, and loop unrolling. The first thing to do is to configure the GEM5 simulator with the *Initial Configuration* provided below:

```
INTEGER_ALU_LATENCY = 1
INTEGER_MUL_LATENCY = 1
INTEGER_DIV_LATENCY = 1
FLOAT_ALU_LATENCY = 4
FLOAT_MUL_LATENCY = 6
FLOAT_DIV_LATENCY = 12
```

1) Enhance the assembly program you created in the previous lab called **program_1.s**:

```
int m = 1;
float a, b;
for (i = 31; i >= 0; i--) {
    if (i is a multiple of 3) {
        a = v1[i] / ((float) m << i); /*logic shift */
        m = (int) a;
    } else {
        a = v1[i] * ((float) m * i);
        m = (int) a;
    }
    v4[i] = a * v1[i] - v2[i];
    v5[i] = v4[i]/v3[i] - b;
    v6[i] = (v4[i]-v1[i]) * v5[i];
}
```

- a. Manually detect the different data, structural, and control hazards that cause a pipeline stall. Report at least 3 hazards in the code (or the pipeline) and fill the following table:

C Line	Corresponding RISC-V Instruction(s)	Type of Hazard	Pipeline Stage	Cause of Stall
if (i % 3)	REM x11, x7, x10 BNEZ x11, not_mul	Data - RAW	ID	The bnez instruction needs to read x11 in the ID stage, but

				rem writes to x11 and won't complete until the WB stage. This creates a Read-After-Write dependency.
if (res != 0)	BNEZ x11, not_mul	Control Hazard	ID/EX	Branch instruction creates uncertainty about next instruction to fetch. The branch condition depends on x11, and the branch target address must be computed. Pipeline must stall or flush if branch is taken.
a = v1[i]/((float) m << i) m = (int) a	fdiv.s f9, f1, f8 fcvt.w.s x8, f9	Data Hazard - RAW	ID/EX	The fcvt.w.s instruction needs to read f9, but fdiv.s writes to f9. Floating-point division has very long latency, so fcvt.w.s must wait for fdiv.s to complete MEM stage.

- b. Optimize the program by re-scheduling instructions to eliminate as many hazards as possible. Manually calculate the number of clock cycles for the new program (**program_1_a.s**) to execute and compare the results with those obtained by the simulator.
- c. Unroll the program (**program_1_a.s**) two times; If necessary, re-schedule instructions and increase the number of registers used. Manually calculate the number of clock cycles to execute the new program (**program_1_b.s**) and compare the results obtained with those obtained by the simulator.

Complete the following table with the obtained results:

Program	program_1.s	program_1_a.s	program_1_b.s
Clock cycles by hand	<u>2228</u>	<u>1570</u>	<u>1404</u>
Clock cycles by simulation	<u>2191</u>	<u>1774</u>	<u>1565</u>

2) Collect the Cycles Per Instruction (CPI) from the simulator for different programs

	program_1.s	program_1_a.s	program_1_b.s
CPI	<u>2.35845</u>	<u>1.9777</u>	<u>1.95381</u>

Compare the results obtained in 1) and provide some explanation if the results are different.

Eventual explanation:

Hand calculations miss real hardware bottlenecks that appear in optimized code, causing larger discrepancies for 1_a and 1_b.

CPI drops from 2.36 to 1.98 (16% gain) then from 1.98 to 1.95 (only 1% more). High CPI and diminishing returns show we've hit the FP division bottleneck so we can't improve further without reducing divisions.